UNIT-II

UNIT - II

Extreme Programming:
Introduction,
Core XP Values
The Twelve XP Practices
About Extreme Programming
Planning XP Projects
Test First Coding
Making Pair Programming Work.

Extreme Programming (XP)

- Extreme Programming (more commonly known as XP). XP is part of the agile movement that focuses on the writing of the software that will implement the required system. This may involve writing Java code, Smalltalk, C++, C#, database tables, XML files, etc.
- XP has been widely misunderstood and is often associated with hacking, a lack of planning, avoiding the creation of documentation and of programmers wildly attacking code while working in pairs. One ex-colleague joined a company a few years ago that claimed to have adopted XP. One of their "rules" associated with XP was that the developers were not allowed to write any comments in their code (they used Java so they had rules that Java doc was illegal).

Core XP Values

These values are presented as:

- Communication
- Simplicity
- Feedback
- Coverage

Communication

- Communication within a software team – the very idea! Whilst this value is to many so obvious as to not require any explanation at all, it is not necessarily adopted within a software project
- Whatever the reason for poor communications, many problems or defects within software systems can be traced back to poor communications during the development of the system. This may be poor communication between programmers, between end users and the development team

Simplicity

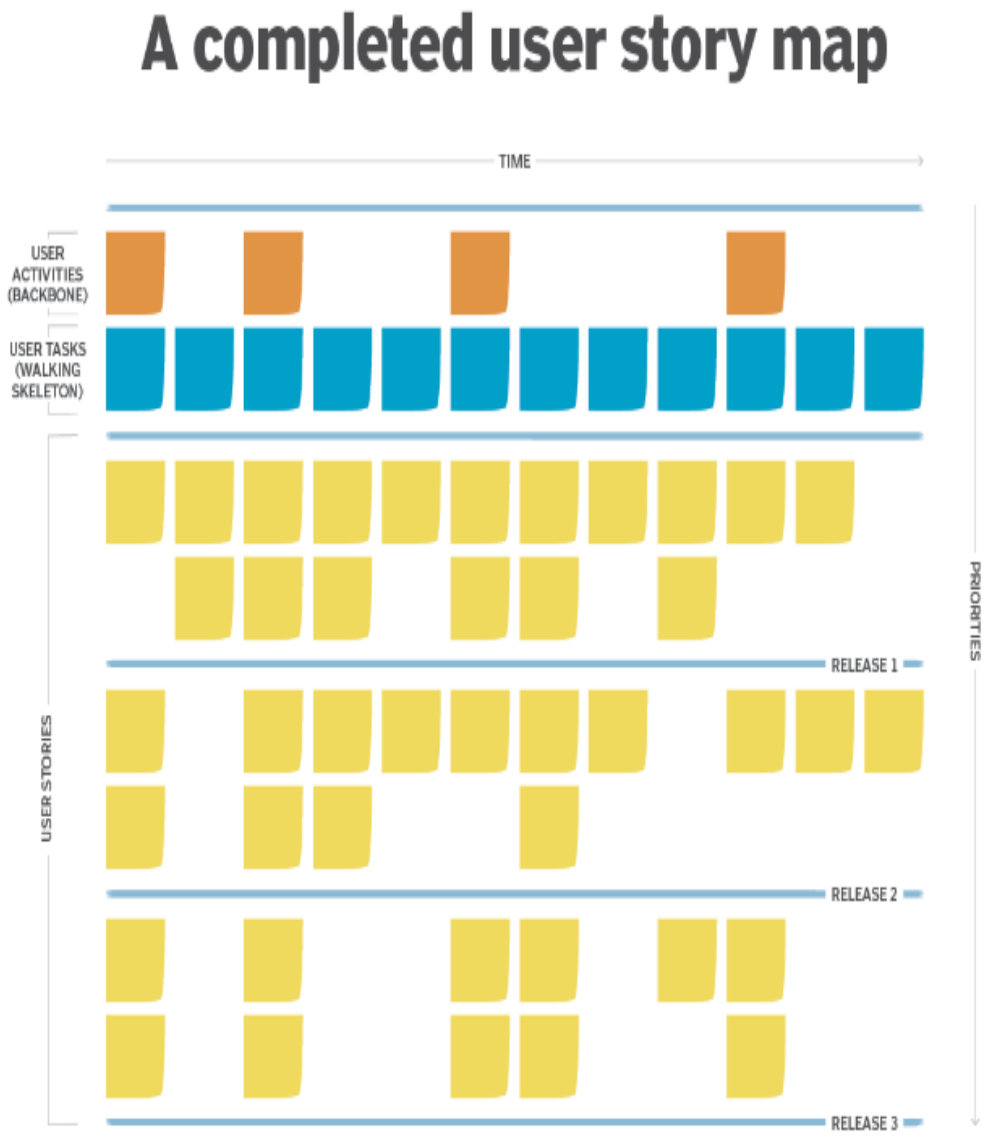
- A rule I have adopted and forced those working with me to adopt for many years now is to *keep things simple*. If a simple solution is adopted, then it is easier for all to understand.
- The simplicity value underlying XP says that you should aim for the simplest solution that does the job (echoes of Agile Modelling here). Why is this, the theory (and general experience is) that the simpler the implementation, the easier it is to implement, test, understand, maintain, etc. Thus, the easier it is to find and correct bugs in the software.

Feedback

- It is good to get feedback. Projects should get feedback early, and often, from the customer, from the team, from real end users, from other project stakeholders, etc. It helps to identify problems early on, deal with unknowns and clarify issues. That is, it generally helps avoid nasty shocks later on.
- Feedback can be at many different levels, for example, by running unit tests every time any new code is integrated into the system, any problem introduced by the new code can be identified immediately.

Courage

You will need courage to adopt XP. Why? Because you need courage to refactor code (that is change existing code so that it is better than it was before but such that it does not provide any new functionality), you need courage to throw away code, you need courage to code for now and leave tomorrow to tomorrow and you need courage to move management to adopt XP's way of working



User stories are developed any time by customers, but particularly during the planning game–a process which occurs at the start of an XP project. A user story is a description in the customers' own words, describing what the system needs to do. Conceptually, they are written by the customers, although in practise a developer may write down what a customer tells them

Each user story has a name, a short paragraph describing the purpose of the story, an estimate of how long it will take to implement (which may be "we don't know"), and a relative importance (such as "must have," "should have" and "niceto have").

Note that it is customers, however, who can validate (and reject) stories and not developers.

User Stories

The Twelve XP Practices

- Given the four values presented above, twelve practices have been developed that try to translate these into a way of working that achieves the aims of XP. If you like, they are the twelve “best practices” that will allow you to fully adopt Extreme Programming on a development project.
- Looking at these best practices, you may wonder at their apparent simplicity, but you should note a number of factors about them
- There are twelve core practices which effectively define XP. If you have not adopted all twelve, then you are not truly doing XP (although you may be very near to XP within some agile continuum).

The 12 practices are:

- *The planning game.* This focuses on planning the next release.
- *Small releases.* A software system is developed iteratively with small releases adding system features and allowing rapid feedback.
- *Simple design.* Keep things as simple as possible but not simpler.
- *Testing.* Unit tests and acceptance tests must be continually developed and the code must pass unit tests for development to continue.
- *Refactoring.* This involves improving the system (e.g., to aid simplicity) with- out changing the functionality.
- *Pair programming.* All code is developed by developers working in pairs (at a single machine).

- *Collective ownership.* Everyone owns all the code so anyone has the right to change any of the code at any time in order to improve it.
- *Continuous integration.* New code is integrated and the system rebuilt every time a task is completed (which may be many times a day).
- *On-site customer.* Have a real customer as part of the team, so that they are always available to answer questions.
- *Coding standards.* Have them and use them.
- *40-hour week.* Work no more than 40 hours a week so that the developers are always fresh and ready for the challenges facing them.
- *System metaphor.* Use the system metaphor to guide the whole development. It is a metaphor for how the system operates (it is similar to the architecture of the system but typically simpler).

What is So Extreme About Extreme Programming?

When you look at the twelve practices that define Extreme Programming or XP, you will probably notice or at least comment on, the fact that there is very little new here. Many of the practices are, or have been incorporated into, described by, or presented as, parts of various existing software engineering methodologies. So what is so different, so extreme, about Extreme Programming? The answer lies in a number of places:

- Extreme Programming is very lightweight – it really only focuses on the programming of a software system.
- Extreme Programming takes the best practices to their ultimate conclusions. Kent Beck says that he pictured each of the practices as a knob on a control board and turned each knob up to maximum and looked at what happened. What happened was XP!
- If we consider the second point in more detail, the concept goes as follows:
- If code reviews are good, then review code all the time (pair programming).
- If testing is good, everybody will test all the time (unit testing), even customers (acceptance testing).

- If designing is good, then make it part of what everyone does every day (refactoring).
- If simplicity is good, then always strive for the simplest effective solution (i.e., the simplest solution that works).
- If architecture is important, then ensure that everyone is involved in creating and refining the architecture all the time (system metaphor).
- If integration testing is good, then integration and testing should be an on going (daily or even hourly) thing (continuous integration).
- If short iterations are good, then make them as short as possible, i.e., hours, or days not weeks and months (the Planning Game).

Planning an XP Project

- XP projects involve a great deal of planning. This may have come as a shock to some as they may have thought of XP as legalized hacking and thus considered planning to have no part in XP. But planning is core to XP.
- Developers begin to examine the plan and start making comments about how anyone could expect them to produce component X is 3 days, or question where the database migration task is.
- As development progresses, developers tell their project leaders of their progress and estimated completion times, projects leaders tell their manager. At each stage, people unconsciously (or otherwise) err on the side of optimism in their estimates. Resulting in a better-estimated position than is the actuality.
- The manager may or may not enter into the plan, the current state of the project, as he sees it.
- At some point, some critical milestone is missed, at this point, the plan is re-assessed and it is realized that the plan and reality are out of alignment. The manager then tries to find a way of squaring the circle of the project. At this point, they may start forcing people to work excessive overtime or may negotiate a later delivery or modify the list of features to be implemented. All of which usually involve extensive discussions with the client.
- The resulting plan may lead the whole project back to step 1 again and a new cycle of catastrophe planning.

Agile Influences on Planning an XP Project

- You are working on an XP project, so naturally you want to plan in an agile manner. That is, you wish to apply agile principles to planning your work. But what does this mean? There are a number of implications for how you will carry out any planning for an agile project that come from trying to be agile.

These are:

- Plan for now
- Responsibility
- Dependencies
- Simplicity

- *Plan for now.* This is akin to only implementing what you need for today and leaving the rest for tomorrow. Thus, you should only do the planning that you need for the next release/next iteration in detail and no more. You can still do long-term planning, but not in such great detail.
- *Responsibility.* The typical project is planned top down. That is, the top-level management try and plan the project (hopefully with input from the actual developers). The end result is a plan for which the developers feel no responsibility and towards which they feel no trust.
- *Dependencies.* When planning an XP project, you should ignore the dependencies between parts. That is, you should not worry about whether task x is required before task y. Instead, you should focus on the highest business priorities first and allow other issues to come out in the wash.
- *Simplicity (in planning).* You should keep in mind the type of planning that is being done at that point in time in order to keep things as simple as possible (but not too simple). For example, if you are planning to help determine the priorities and ordering of user stories, then much less detail is required.

The Planning Game in Detail

- The planning game relies on user (customer) stories to drive the iterations of the project. These user stories are used to determine which features will go into which iterations. From this, appropriate releases can be identified.

Although the primary output of the Business Game is the plan, it:

- allows customers to make business decisions,
- allows developers to make technical decisions.

And to combine the results into the iteration oriented plan. Within this framework the customer determines:

- Scope – what is in and out of the system.
- Priority – what is more important, less important, must haves and nice to haves, etc.

Composition of releases – what will be in each release. Release dates .

The development team decides

- Estimates of how long various features will take.
- Consequences of for example, using a particular technology, for example,
- Linux versus Microsoft Windows XP, Java versus C#, J2EE (Java 2 Enterprise Edition) versus .Net.
- Team and Project Organization (e.g. the organization of the tasks) Risks associated with different features (e.g. will MySQL provide the required level of performance or should an Oracle database be used). Detailed scheduling – which features are to be done when and in what order.

- The game has three phases through which play proceeds. These phases are the exploration phase, the commitment phase and the steering phase.

Elaboration phase

This phases help to identify what the system needs to do. It is compressed of the following steps:

- *Write a story* – the business writes a user story describing some behavior required of the system.
- *Estimate a story* – development estimates how long the story will take to implement. If they cannot estimate the story or the story seems too big then they can ask for clarification or break the story up into smaller chunks.
- *Break a story up* – the business must break a story down into small chunks if required.

Commitment Phase

- This phase allows business to determine the scope of the release and when that release will occur (based on information provided by development and business). The steps within the commitment phase are:
- *Sort by value* – business must sort the stories (written on index cards) into three plies (1) must have, (2) should have and (3) nice to have. This is effectively applying a relative priority to each story.
- *Sort by risk* – Development now sorts the stories into three further piles (1) stories that can be estimated precisely, (2) stories that can be estimated roughly and
- (3) stories that can't be estimated.
- *Set velocity* – development indicates how quickly they will be able to achieve the estimates (which have been made in an idealized type of time).
- *Choose scope* – Business selects which user stories will be in the next release.
- **Steering Phase**
- The purpose of the steering phase is to allow the plan to be updated as things change over time. This phase is a little different from the last two in that the last two are likely to have happened at the same time

- *Iteration* – from each iteration the business picks the stories that will form that iteration.
- *Recovery* – if it is realised that not all the stories can be implemented for a particular iteration, then Development must ask Business to help determine which stories should remain in the iteration and which might be moved to later iterations.
- *New Story* – if Business

By following through the planning game, a set of requirements for each iteration can be produced. Depending on the stage of the project, the current iteration may be planned in detail, with a future iteration only penned out in terms of anticipated user stories. This is okay as at the start of each iteration, the planning game will occur again. In fact, there are actually two types of planning games, they are:

- The initial planning game
- The release/iteration planning game.

Planning XP Projects

The XP project lifecycle is presented in Figure 6.1. This diagram illustrates the various planning stages and implementation stages within a typical XP project.

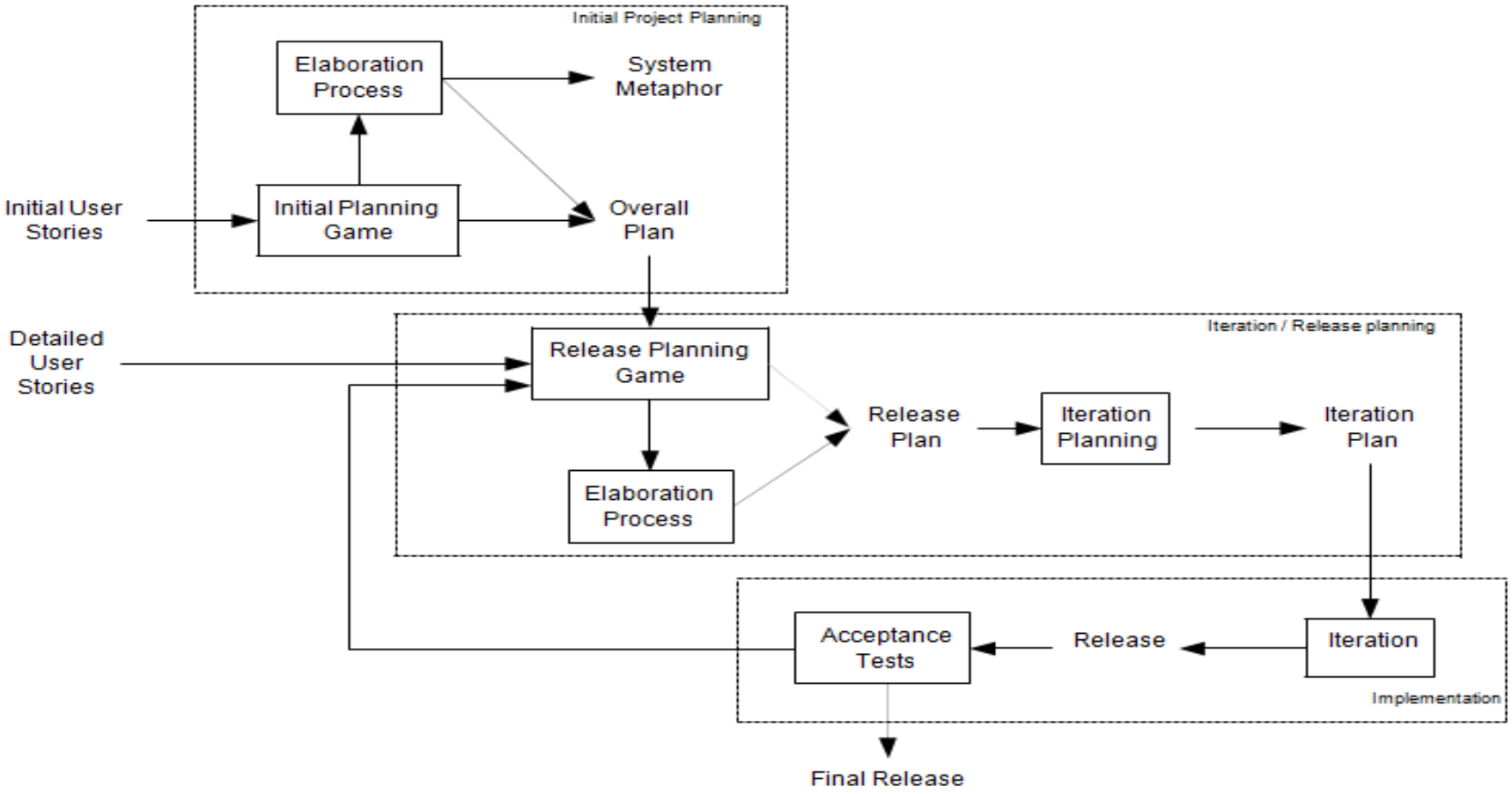


Fig. 6.1 XP project lifecycle.

- For example, it starts with an initial project planning process during which the overall plan of the project is roughly sketched out. This is followed by one (and typically more than one) release planning process where the contents of a release is planned and the tasks performed in the iteration to implement the release are also planned.

1.Playing the Planning Game

- The planning game has only two conceptual players; these are the "Business" and "Development." The business is formed from those stakeholders who can specify the operation of the system. Development is formed from members of the development team relevant to the features being discussed.

2.The Goal of the Game

- The goal of the planning game is to maximize the value of the software produced by the team.

3.The Strategy

The main strategy of the game is to invest as little as possible to put the most valuable functionality into production as quickly as possible, but without com- promising the required product quality.

1. The strategy is to “invest as little as possible.” That is, to get the job done as quickly as possible without incurring unnecessary overheads. For example, the idea of not implementing features today which may or may not be required tomorrow.
2. The aim is to “put the most valuable functionality into production.” Few projects have unlimited resources and unlimited time.
3. We want to do all this “without compromising the required product quality.” Obviously, we want the software we produce to do its job without causing the users any difficulties.

4.The Game Pieces

The pieces used by the two players are the “user stories” described in the last chapter. These are written down on index cards and moved around during the game. Of course you do not necessarily need to use index cards, user stories can be written down on white boards, scrapes of paper, entered into a software system for recording and reference.

The Players

As mentioned earlier, there are only two players in the game “Business” and “Development.” I like to think them as teams, because they are rarely a single person (apart possibly from very early on in the life of the project).

5.The Moves/Playing the Game

There are three stages to play the game.

- *Exploration* – Determine new user stories of the system.
- *Commitment* – Decide which features will be addressed in a particular release.
- *Steering* – Update the plan as development progresses.

- *Exploration Phase*
- Within the exploration phase, the game tries to help the team identify what the system should do. To achieve this, this phase has three steps (or moves). These are:

- Write a story.
- Estimate a story.
- Split a story.

Commitment Phase

During this phase of the game, Business must identify what will be in the current iteration and when the next release will be. For their part, Development must commit to the agreed duration and the content of the release. If this situation cannot be met then either the timescale of the release must be changed, the content of the release altered or the number of developers increased.

The steps presented to reach agreement within the game are:

- Sort by value.
- Sort by risk.
- Choose scope.
- Set project velocity

Steering Phase

In the real world, plans often change. This may be for a wide variety of reasons including (but by no means limited to):

- changing requirements,
- new requirements,
- changing priorities,

incorrect estimates,

changing resources (developers leave and new developers join a project with different skill sets

The steps in the steering phase are:

- Iteration planning.
- Project recovery.
- Identifying a new story.
- Project re-estimation.

Planning Your XP Project

- So how do you use the planning game to plan you XP project? What do you do when and how? In this section, I will look at how the various types of planning game fit together to help you plan your XP project.
- There are actually two forms of the planning game:
- The initial planning game, and
- The release planning game.

- Thus, a typical XP project might be planned in the following manner:
  - An initial planning game (aims to get overall view of project);
  - Initial elaboration process (focusing on high level user stories);
  - Release 1 planning game;
  - Release 1 elaboration process (if required);
  - Plan iteration 1;
  - .... Release 1 iteration/implementation ...;
  - Release 2 planning game;
  - Release 2 elaboration process (if required);
  - Plan iteration 2;
  - ... Release 2 iteration/implementation ...;
  - . . .
  - Release *n* Planning game;
  - Release *n* Elaboration process (if required);
  - Plan iteration *n*;
  - ... Release *n* iteration/implementation.
-

- *The Initial Planning Game*

  The initial planning game focuses on what the system as a whole should do. It considers all user stories that are in scope (and indeed what that scope is). It happens at the start of the project and may reconvene at various points throughout the lifetime of the project to review the scope of the system, the set of user stories, their relative priorities, etc.

- *The Release Planning Game*

  The release planning game focuses on the contents of a release or iteration. It has the same steps as the initial planning game but the level of detail that needs to be considered is much greater. During the initial planning game, the overall plan for the project should have been roughly planned out. At the start of a release, the details of what will be done in that release need to be determined. This means that:

- User stories need to be fleshed out and may need to be broken down into finer grained stories (in order that they can be implemented).
- Detailed estimates of the stories need to be obtained.
- The user stories to be implemented as part of the release need to be confirmed, revised or modified as required.
- The project velocity may need to be revised. For example, as the development team become more experienced in XP and the application in which you may find development speeds up.

- *The Elaboration Process*
- The elaboration process follows the initial planning game, and typically on a smaller scale, a release planning game. During this phase, research is carried out to clarify user stories in order to estimate, clarify requirements, or technical issues. The aim of this is to:
- Lower the risk of bad estimates,
- Experiment/prototype different solutions,
- Improve the development teams to understand the domain/technology,
- Ensure procedures and processes required are in place.

- *Iteration Planning*

There are two issues to consider when planning an iteration. These are (1) determining the size of an iteration; (2) determining what should be done within an iteration to implement the user stories.

Size of an iteration. Ho do you determine how big an iteration should be? How long is a piece of string? The answer is that an iteration needs to be big enough to allow either a new release to be created that adds value to the business or large enough that you are able to make significant progress.

Planning the iteration. If the release planning game identifies what new features should be added to the evolving system in the current iteration, then the iteration plan defines how those features will be achieved.

- Iteration planning usually incorporates the following phases:
- Evaluation of the last iterations lessons learned, changes to be made, etc.
- Review of user stories to incorporate into the iteration.
- Task exploration during which tasks are written for user stories.
- Task commitment during which tasks are estimated, load factors determined and balancing takes place.
- Finally, the iteration plan is verified

Test First Coding

How to Write Tests First?

So how do you move to an approach in which you write the test first? There is no hard and fast answer, as this question is a bit like "so how do you decide what to program first." But the following points try to give a flavour of how I (and many others) try to go about test first coding.

1. Think about what the code should do and try to ignore, for the moment, how it will do it. This can be very difficult for programmers to get to grips with, not least because in general their focus has always been on "how." It can feel like a leap of faith, or a bit like wandering around in the dark (with your eyes closed!).

2. Now write a test that will use the classes and methods you haven't yet implemented! This will also seem strange at first. How can you write a test using code that doesn't yet exist! It is possible.
3. If you haven't already done so, then write the stub code for the classes being tested. As mentioned above, tools such as Eclipse may do this for you, however, if you are using tools such as Emacs you will have to do it yourself.
4. Now put the code you have created for the test into whatever project code repository you are using. This includes your test code. If you are using a framework such as JUnit then add your test to that as well.
5. Now run your newly created test against the stub code. It should fail but that's okay – you haven't implemented the methods you are testing yet! This is actually an important step (although at this point you may think it redundant).

6. Now you are in a position to actually write the implementation of the methods being tested. This may lead to creating additional supporting methods but that's okay. However, your focus should be on writing only enough code required to pass the test. This firstly focuses your efforts on exactly what is needed; ignoring additional bells and whistles that you think might be useful in the future. It also helps you to produce the simplest code that meets all the requirements. After all, if it passes all the tests then it meets its requirements no matter how simple the code is.
7. Rerun your tests against the newly implemented methods. If all the tests pass, then continue. If not returned to step 6 and revise your code as necessary.
8. Now re-run the tests for the entire system. If all tests pass, then add the changes in your current test suite and implementation to your code repository (your tests may have needed revision as you implemented the classes and methods). Now you can continue. If one or more of the tests fail then you must return to step 6 and revise your code as necessary (as it must have been your changes that caused any tests to fail).
9. Refactor your code for clarity and to remove any duplication. Return to step 7 if you are sure that no refactoring is required, then you have finished the current test. Return to step 1 for the next test.

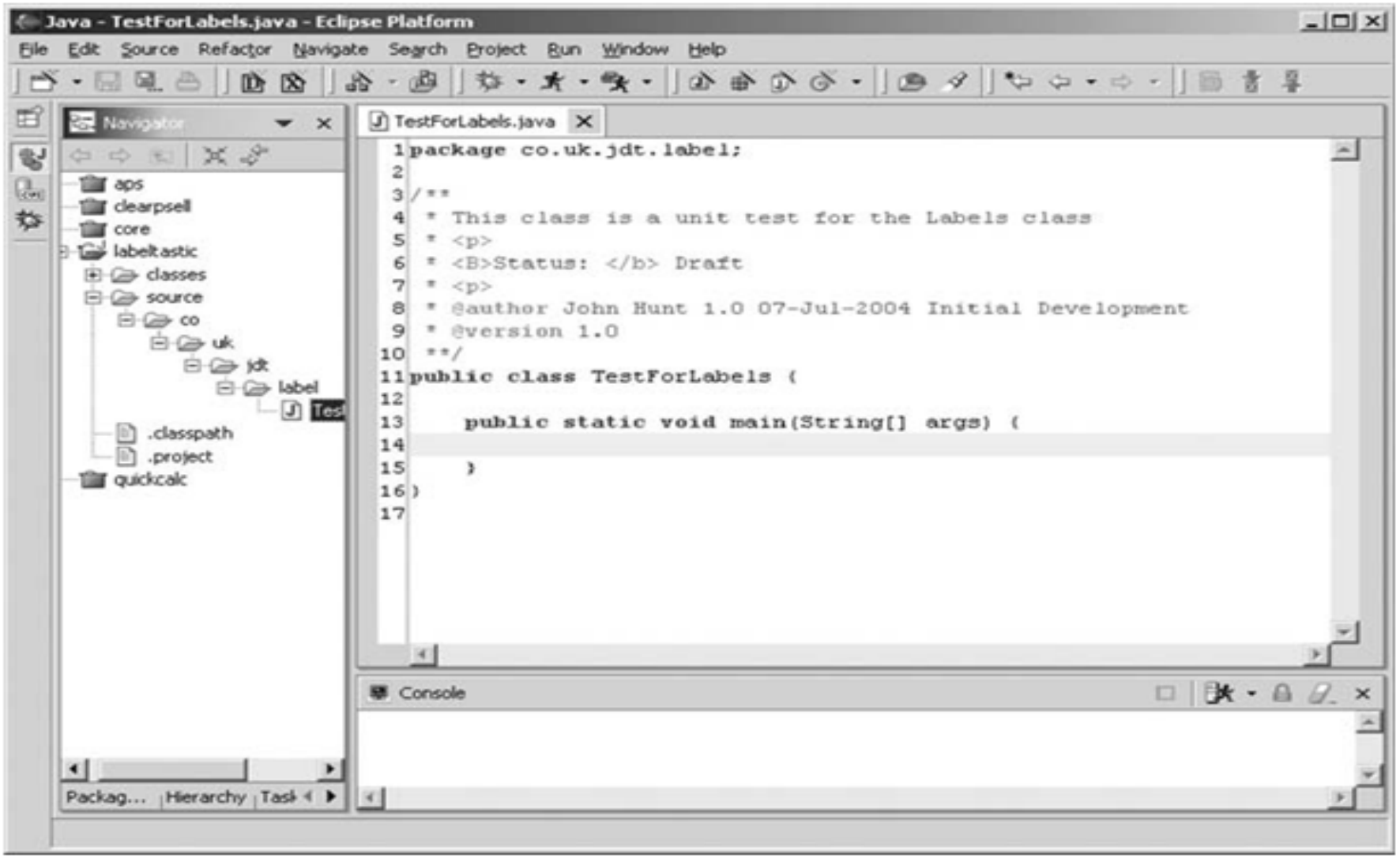


Fig. 6.2 Creating a simple test in Eclipse.

For example, in Figure 6.2, I have started to create a simple test class called *TestForLabels* using Eclipse. At this point, there is no behaviour defined for the main method in the test case

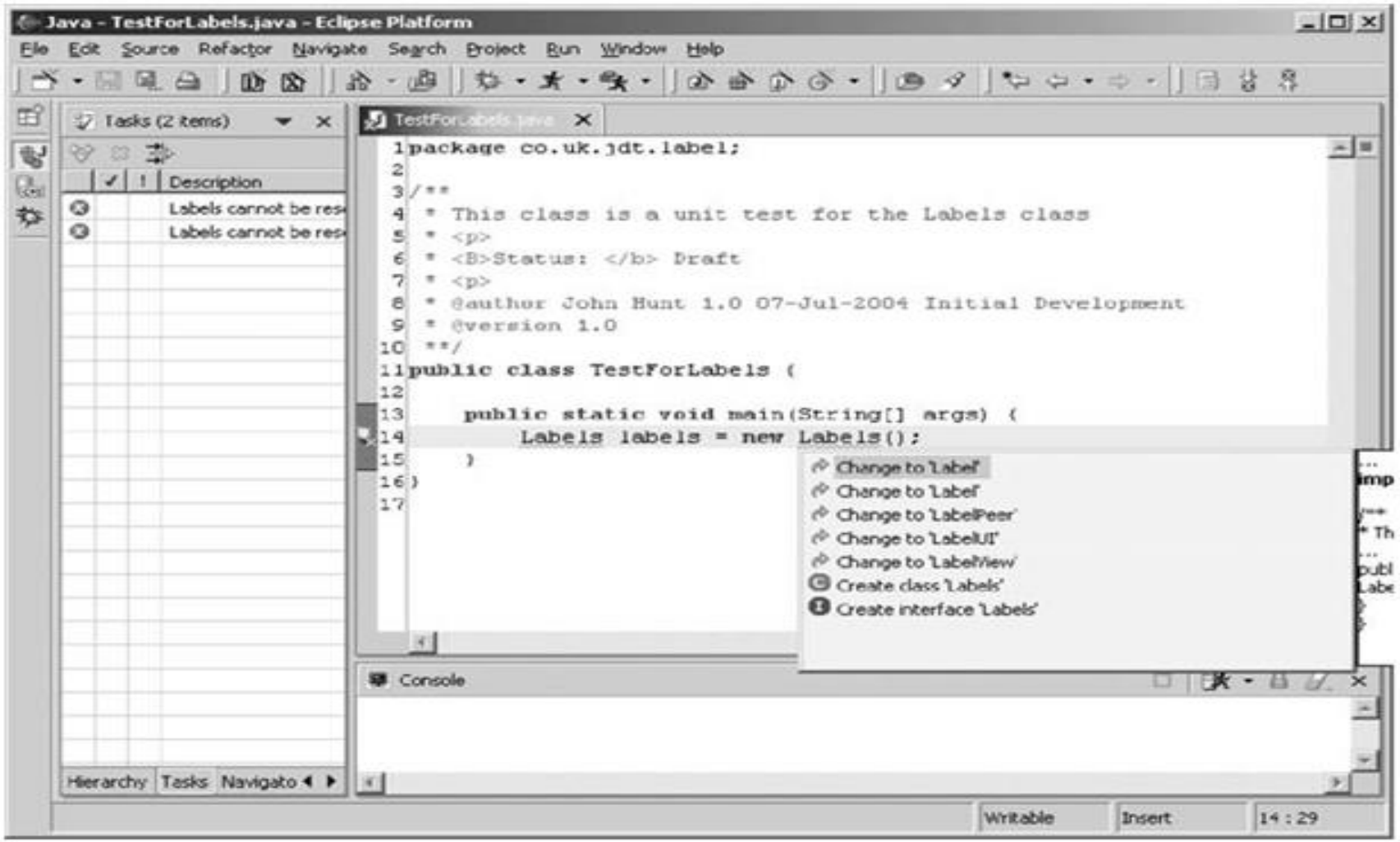


Fig. 6.3 Eclipse offers to create an as-yet undefined class.

In Figure 6.3, I have added a statement that creates an instance of a class that I have yet to define. Eclipse notices this and prompts me with a set of options. Two of these options allow me to either create an interface or a class. In this case *Labels* is a class that I want to instantiate (and interfaces cannot be instantiated). Thus, I will select the option allowing me to create the Labels class (at the click of a mouse button!).

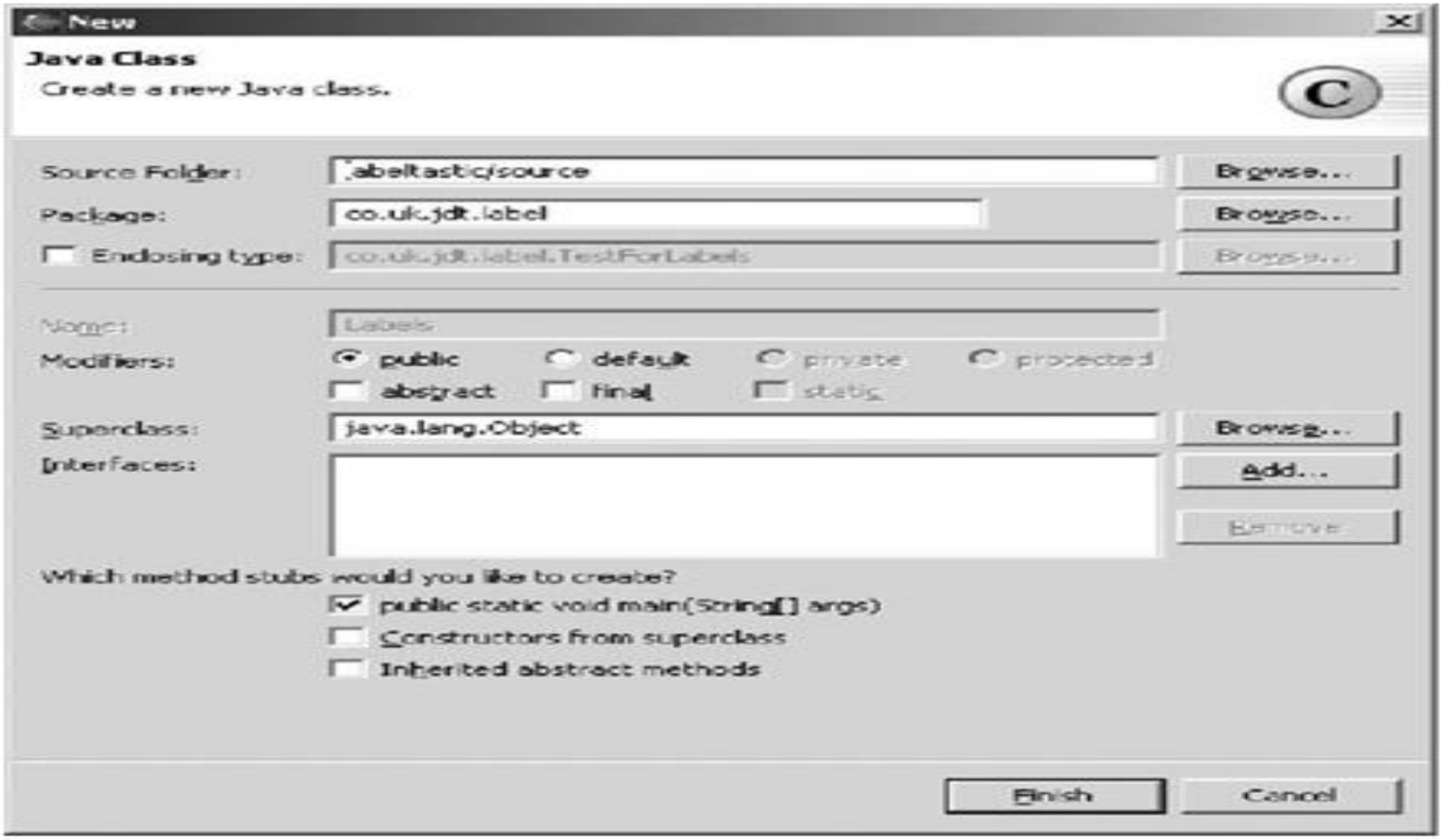


Fig. 6.4 The Eclipse class creation dialogue.

Figure 6.4 illustrates the class creation dialogue presented by Eclipse to allow me to create a new class Labels. It allows me to select the parent class, interfaces to implement and whether I want any abstract methods to be implemented and constructors provided. Using this feature, I can quickly create many of the stubs I would need for a new class.

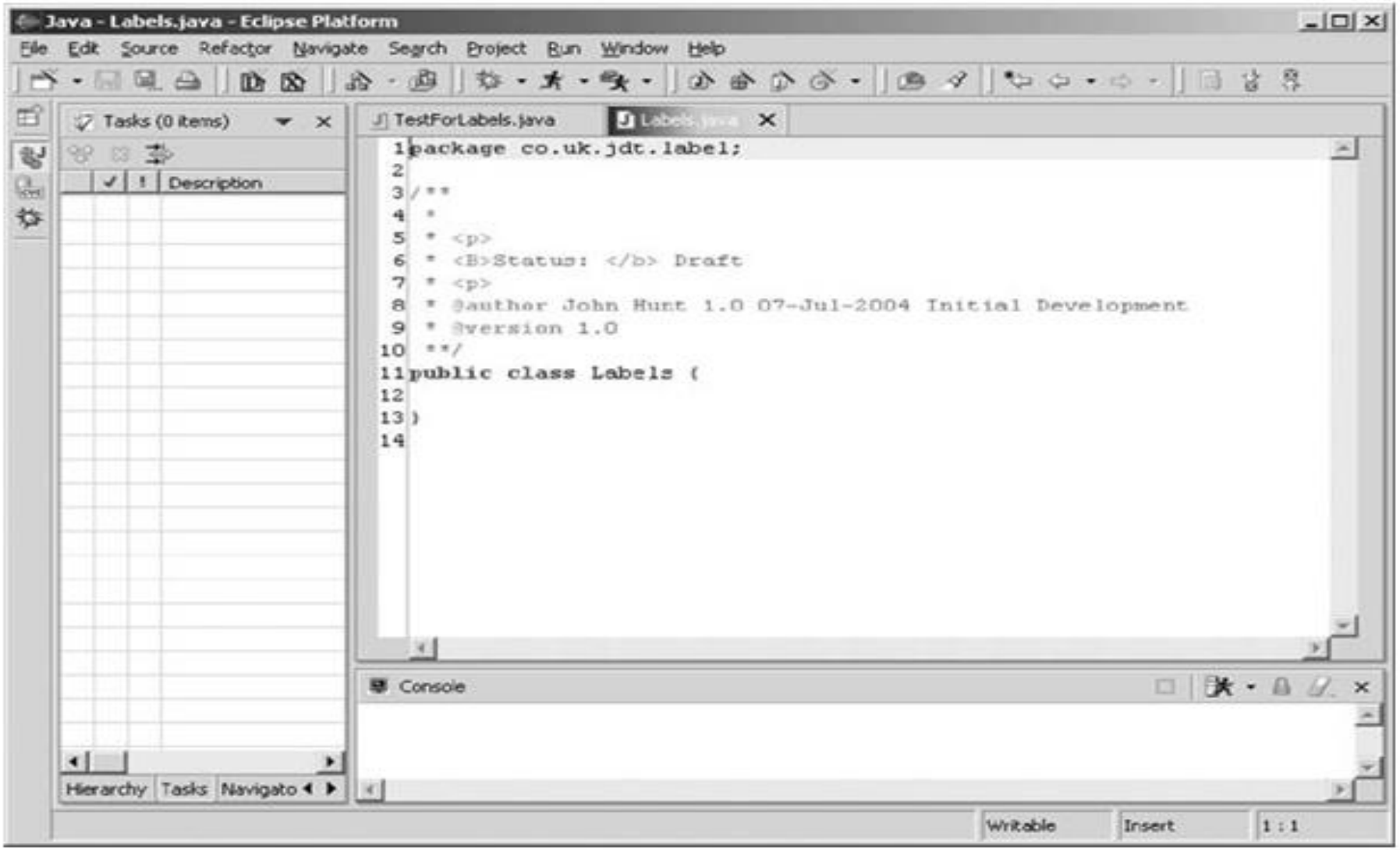


Fig. 6.5 The initial "implementation" class with no methods.

Figure 6.5 illustrates what the (very simple) resulting class looks like in Eclipse. At the moment, this class will compile but has no methods (as I am keeping things simple).

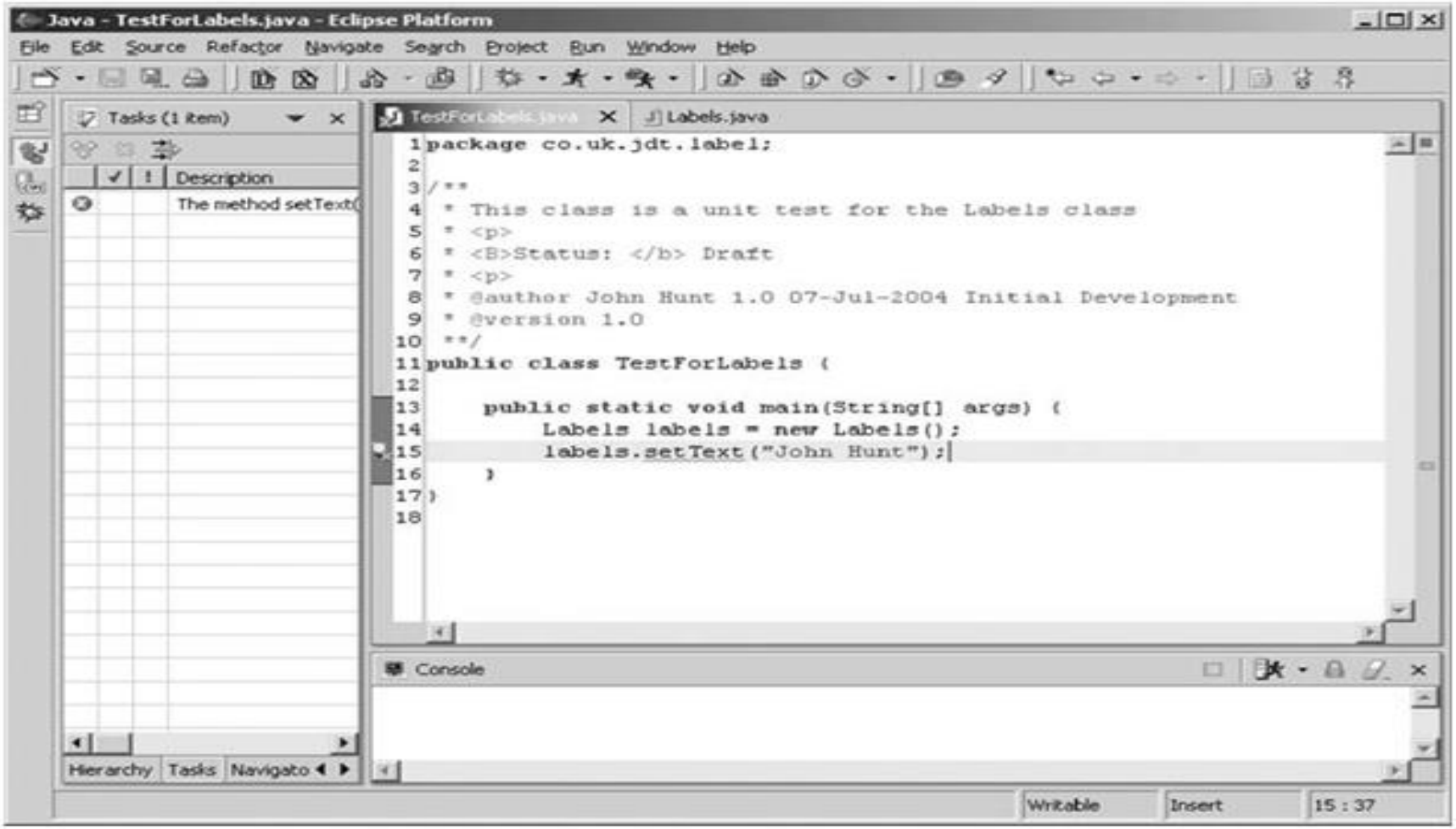


Fig. 6.6 Adding some behaviour to the test class.

I can now return to my test case and add some more to it. In Figure 6.6, I have added the following statement:

```
labels.setText("John Hunt");
```

This calls the method *setText* on the newly created object referenced by the local variable *labels*. But *setText* does not yet exist. I have just referenced it within the test harness.

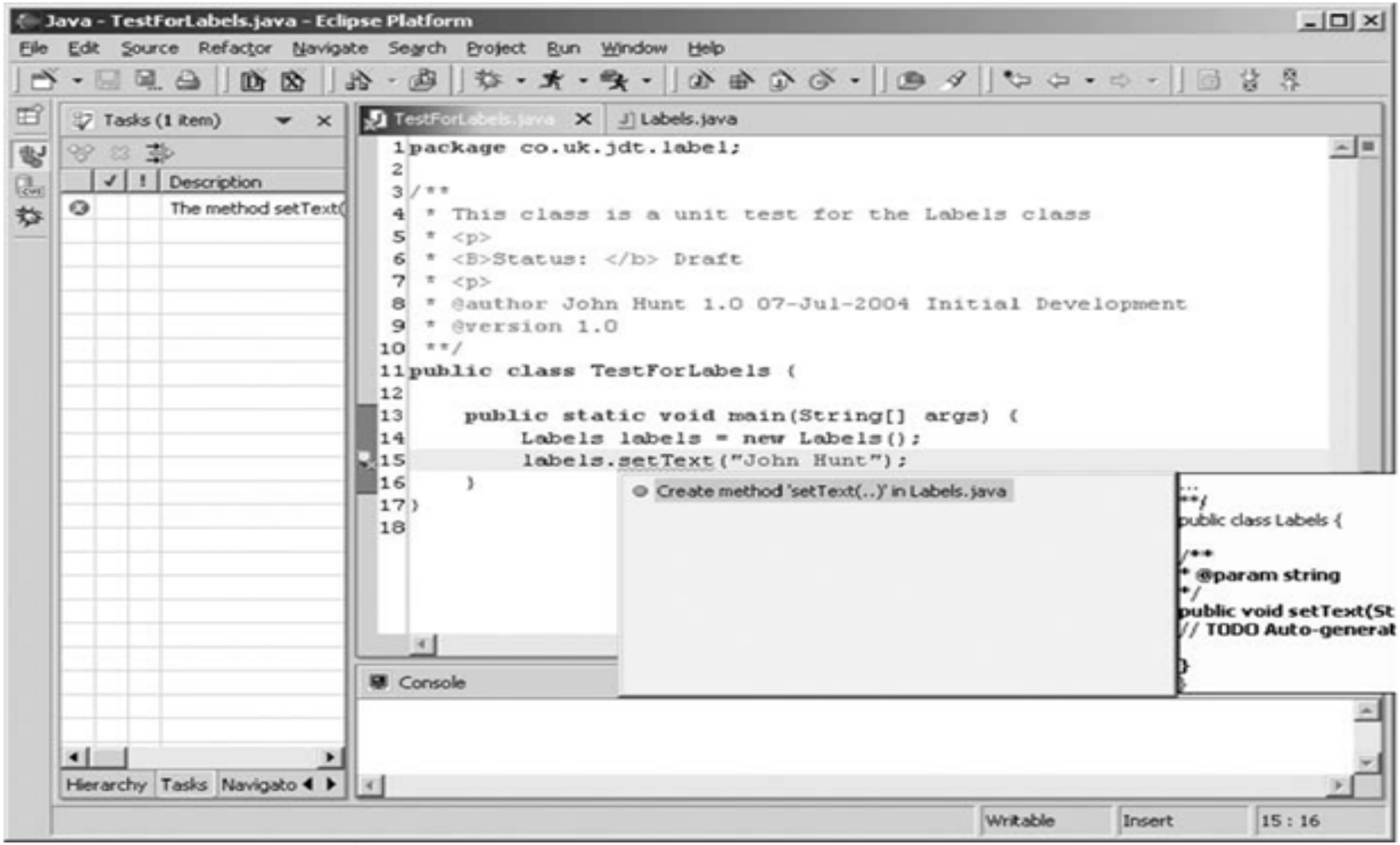


Fig. 6.7 Eclipse prompting to create the undefined method.

Again Eclipse notices this and prompts me to allow it to create this method for me. This is illustrated in Figure 6.7. It even gives me a preview of what it would generate in a box to the side of the editor.

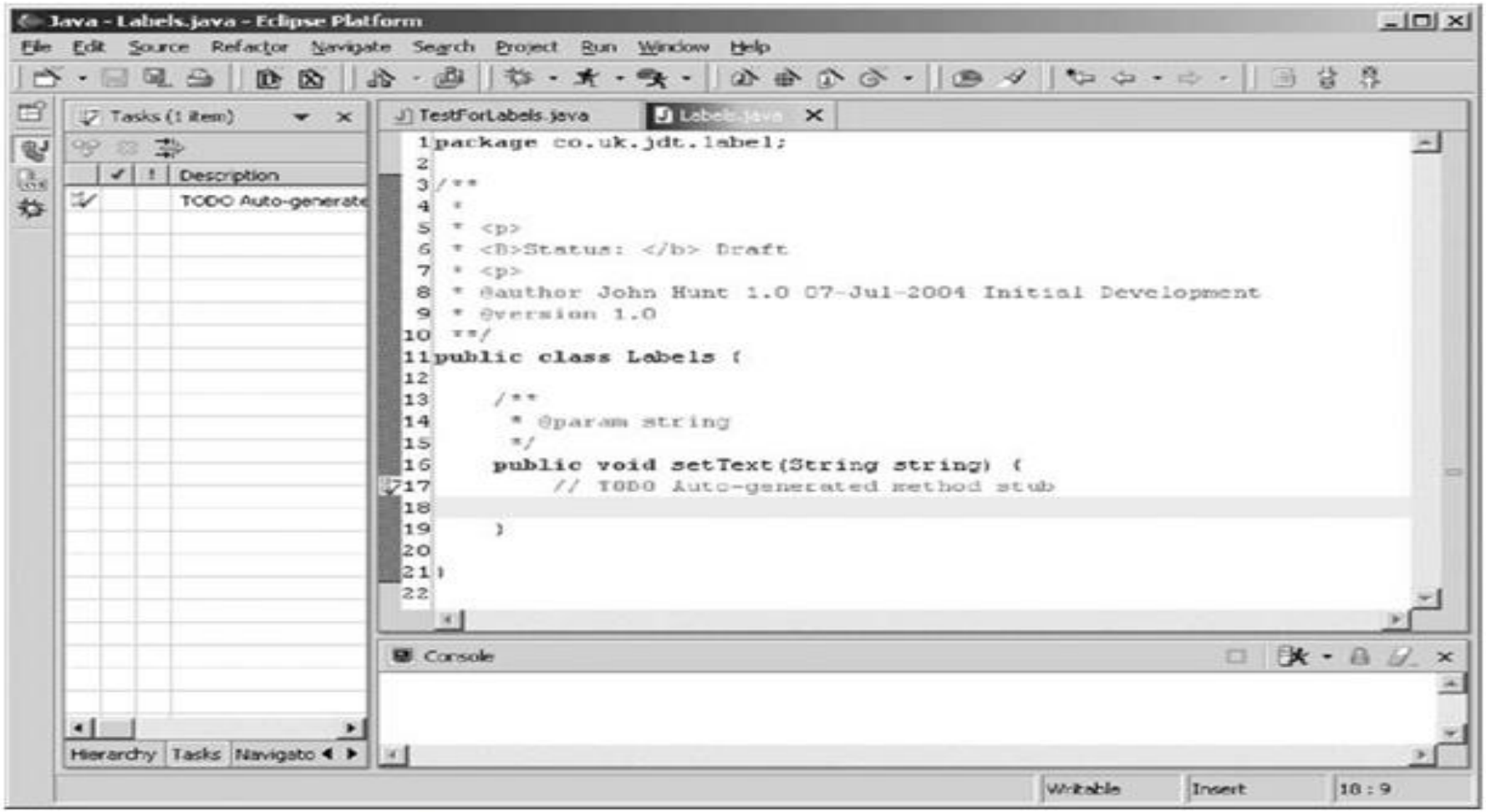


Fig. 6.8 The method auto-created by Eclipse.

The result of Eclipse auto-creating the *setText* method is presented in Figure 6.8. This method takes a string as a parameter and does nothing with it. But that's okay; it is enough to allow the test harness to compile. We can take this further by entering another method into the test case example requiring the text to be returned for the label, using a *getText* method

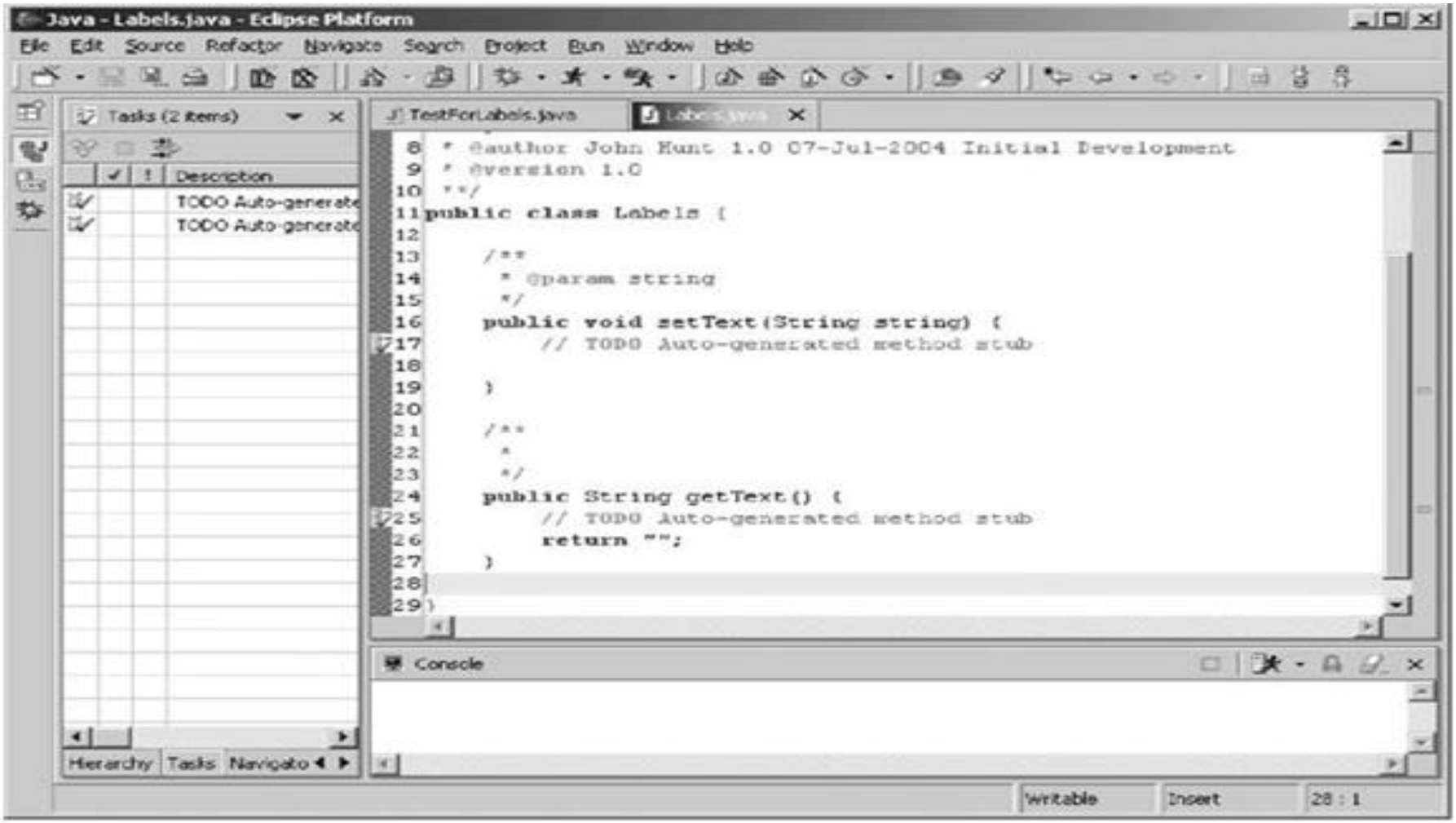


Fig. 6.9 The class with two auto-created stub methods.

. The result of Eclipse auto-creating this method is presented in Figure 6.9. Note that this method re- turns a value, thus a default value is defined in the return statement. This will not be correct in all situations (and indeed should not be correct in all situations). This is fine, as it is enough to allow us to compile the test class.

What to Test?

Okay so testing is good and writing tests first is better. Great, but what should I test? Should every single method in every class in my application have a test written for it? Probably not! Should I only write tests for major subsystems – again probably not! So, what should I test?

As with many things in software, there are no hard and fast rules for what you should test, but here are some XP-oriented guidelines:

- Write tests for any tasks being implemented.
- Write tests for any classes or combinations of classes that are non-trivial and could easily be broken. For example, if one particular class implements a complex algorithm to find sneak circuits within electronic relays, then write tests for that class even though it may only be one element of a larger module.
- Avoid writing tests for methods that just call another method, e.g. delegate methods, if the called method already has a test written for it.
- Assume that more tests are better then less and that no one will complain about having too many tests, but that people will complain if an important test is missing.
- Write tests that instil confidence in the system. For example, if one area of the system is invoked by many others (such as a data access management subsystem), then having a set of tests for that area adds to the confidence of those using it.
- Add tests that will help cover areas of the system being modified. If you are refactoring some code, and feel that one or more tests are missing then add them.
- If you happen to notice that a suite of tests for a particular module, subsystem or class appears to be missing in one or more tests, then add them.

Making Pair Programming Work

The idea behind pair programming is of course very simple, essentially it comes done to “two heads are better than one” most of the time. In addition, in pair programming all code is always reviewed by at least one other person who is focussed on what the code needs to do.

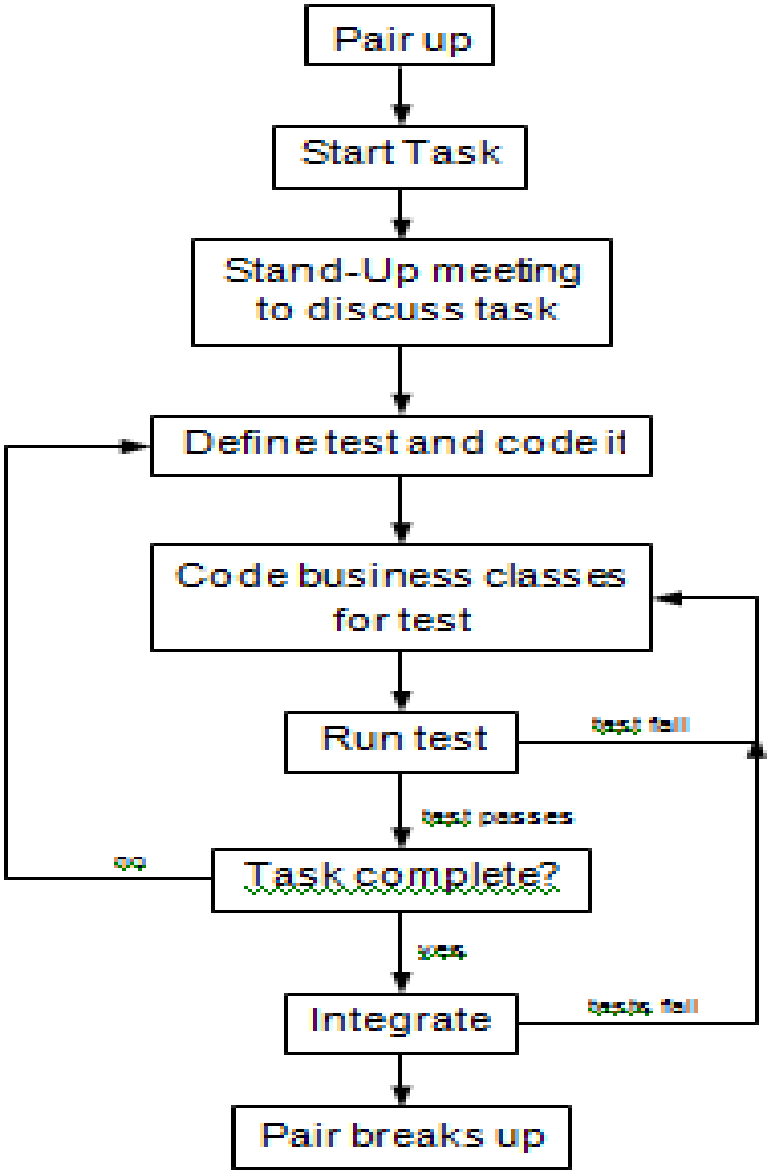


Fig. 6.10 Pair-programming workflow.

The typical pair programming workflow is illustrated in Figure 6.10. It illustrates a number of important features of pair programming. Firstly, pairs are not permanent (and indeed should be changed regularly). A pair forms to carry out a particular task.

- At the start of the task, they have a brief meeting to discuss what it is they are about to do and how they might approach it, what the options are, etc. This helps to ensure that no one is along just for the ride.
- They then work on the code in a test first manner, proposing a test, implementing the business classes that will meet the requirements of that test and then running the test.
- If the test is passed, then they move onto the next test.
- If the test fails, then they review the code and determine what is wrong.
- Once the tests are completed, they integrate the code with the current build. If this causes any tests for the whole system to fail, they must revise the code and determine why the test failed, etc.
- They can only finish integrating once all the tests have passed
- The task is then completed and the pair breaks up.

Whilst there are no hard and fast rules to how to do this, there are things that can help:

Pair programming takes practice. Ideally, in any pair, one of the pair should be an experienced pair programmer. In many situations, this is not possible, so the Below points should be borne in mind by all members of an XP team. The key thing is to stick with it.

1. *Engage in a dialogue.* The “driver” should try and explain what they are doing. The “navigator” should ask questions in order to understand what is being done. This is not as simple as it sounds. For example, the next time you are in your car and driving, try telling an imaginary passenger what you are doing, what you are considering and what concerns you have about the road ahead (it helps if the passenger is imaginary so that none of your friends think you are mad!).

2. *Listen to each other*. If one member of the pair is doing all the talking then it is probably isn't working. The one who isn't saying much might not be clear on what is being done. Try swapping roles, or have a break, or take time out to review where you are.
3. *Take frequent breaks*. Pair programming is intensive. It is intensive in terms of the little grey cells and also in terms of inter-personal communications and dynamics. Have regular breaks, talk about other things, see how others are getting on, catch up on the news, etc. Don't take a break and discuss the code!
4. *Make pair programming practical*. Providing enough space to allow a pair to work together comfortably isn't essential but it can help. This may only go as far as having desks that are big enough for two developers to sit next to each other, or it may involve special double size workstation environments.

5. *Don't be a back seat driver*. Nothing can be quiet as annoying as a back seat driver (particularly if its your mother in law!). One of the common problems, particular if the "navigator" is more experienced relative to the current task than the "driver" is the back seat driver syndrome. If you are the navigator and you find yourself telling the driver what to do all the time try and stop yourself. It is not only really annoying but it is not pair programming. You are not performing the role of the navigator. You are acting as the virtual driver and using someone else's fingers to type the code in! You can either swap roles, or better stop and discuss your proposed solution with the driver and then let them run with it. They will learn a lot more.
6. *Use a common environment*. Developers tend to have their own personal favourite development environment. For Java this might be JCreator, JBuilder, NetBeans, Eclipse or even Emacs, etc. However, having different development environments makes pair programming harder. Selecting one development environment to be used by all makes life a lot easier.

7. *Shared language and vocabulary*. Developers who have a shared vocabulary of design and programming concepts tend to work together better. This is partly because it avoids the "what do you mean by *Factory Method*?" type question. This is not as easy as it may seem. As an analogy, consider the following question that I was once asked when buying some fast food while visiting the USA:

- "Would you like that on a biscuit?"
- To me this was somewhat surprising as what I pictured at this point was what I would call a Digestive biscuit.

8.Allowing non-pair time. There are some situations where allowing a pair to work alone can be beneficial. This may be a little controversial for some in the XP community, as they will argue that it is *always* between to work in pairs. However, in the following situations, flying solo may be an alternative (but it should be the rare exception to pair programming):

- Exploring completing alternative solutions,
- Following multiple competing lines of investigation during debugging (but not bug fixing).

9. *Change partners often*. Pairs should not be permanent. Instead, pairs should change as and when required. The typical point at which to change is at the start of a new task. However, if a task ranges over a number of areas, then a "driver" developer may pair with several "navigators" in order to benefit from their different areas of expertise.

Questions on UNIT-II

- What is Extreme Programming (XP) and when was it developed?
- Who is the primary creator of XP?
- In what types of projects is XP most effective?
- How does XP differ from other Agile methodologies?
- What problems in traditional software development does XP aim to solve?
- What are the five core values of XP?
- How does the value of **Communication** impact XP practices?
- Why is **Simplicity** emphasized in XP?
- How do **Feedback** and **Courage** contribute to project success in XP?
- Explain how **Respect** among team members supports the XP process.
- What are the twelve practices in Extreme Programming?
- Explain the concept of **Pair Programming** with a work flow.
- How does **Test-First Development (TDD)** improve code quality?
- What is **Continuous Integration**, and why is it essential in XP?
- Describe the importance of **Collective Code Ownership**.
- How does **40-Hour Week** relate to sustainable development?
- What is the purpose of **Metaphor** in XP?
- How does **Refactoring** contribute to code quality in XP?
- What are the main goals of Extreme Programming?
- How does XP handle rapidly changing customer requirement

- What are the typical roles in an XP team?
- How is customer involvement ensured in XP?
- What are some limitations or criticisms of XP?
- What is the role of **User Stories** in XP planning?
- How are release plans and iteration plans created in XP?
- What is **Velocity** in XP and how is it used?
- Describe the concept of **Planning Game** in XP.
- How are priorities set in XP project planning
- What is **Test-First Coding** or **Test-Driven Development (TDD)** in XP?
- How does writing tests before code benefit the development process?
- What are the steps involved in TDD?
- How do developers handle test failures in Test-First Coding?
- What is the difference between unit tests and acceptance tests in XP?
- What is Pair Programming and how is it implemented in XP?
- What are the roles of the **Driver** and **Navigator** in Pair Programming?
- What are the benefits of Pair Programming?
- What challenges might teams face with Pair Programming?
- How can teams ensure effective communication during Pair Programming?